

Diagrama de Arquitectura de Microservicios

Proyecto: Tienda Retail Lumina **Asignatura:** Desarrollo FullStack III (DSY1106)

1. Diagrama general (Mermaid)

Este diagrama se renderiza automaticamente en GitHub.

```
flowchart TD
    User([Usuario / Navegador])

    subgraph Cliente
        FE["Frontend SPA<br/>React + Vite + Tailwind<br/>:5173"]
    end

    subgraph Gateway
        BFF["BFF Gateway<br/>Spring Boot + WebClient<br/>:8080"]
    end

    subgraph Microservicios
        US["ms-usuarios :8081"]
        PR["ms-productos :8082"]
        BO["ms-bodega :8083"]
        CA["ms-carrito :8084"]
        DE["ms-delivery :8085"]
        PA["ms-pagos :8086"]
    end

    subgraph Persistencia["Persistencia (Database per Service)"]
        DBU["db_usuarios<br/>MySQL :3307"]
        DBP["db_productos<br/>MySQL :3308"]
        DBB["db_bodega<br/>MySQL :3309"]
        DBC["db_carrito<br/>MySQL :3310"]
        DBD["db_delivery<br/>MySQL :3311"]
        DBPA["db_pagos<br/>MySQL :3312"]
    end

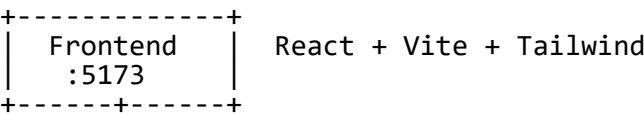
    MP{{MercadoPago API}}

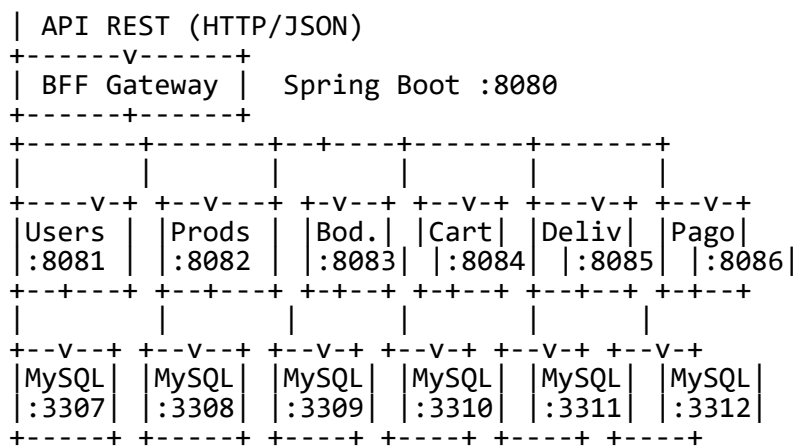
    User --> FE
    FE -- "API REST /api/* (HTTP/JSON)" --> BFF
    BFF -- REST --> US
    BFF -- REST --> PR
    BFF -- REST --> BO
    BFF -- REST --> CA
    BFF -- REST --> DE
    BFF -- REST --> PA

    US -- JPA --> DBU
    PR -- JPA --> DBP
    BO -- JPA --> DBB
    CA -- JPA --> DBC
    DE -- JPA --> DBD
    PA -- JPA --> DBPA

    PA -. SDK .-> MP
```

2. Diagrama ASCII (alternativo)





3. Componentes

Capa	Componente	Tecnologia	Responsabilidad
Frontend	SPA	React, Vite, Tailwind	Interfaz de usuario
Gateway	BFF	Spring Boot, WebClient	Punto unico de entrada, enrutamiento
Backend	6 microservicios	Java 17, Spring Boot 3.2, JPA	Logica de dominio
Datos	6 MySQL 8.0	Database per Service	Persistencia aislada
Externo	MercadoPago	SDK Java	Procesamiento de pagos

4. Patrones de arquitectura aplicados

- **Microservicios**: dominios desacoplados, despliegue y escalado independiente.
- **BFF (Backend For Frontend)**: una unica puerta de entrada para el frontend, que oculta la topologia interna y centraliza CORS y enrutamiento.
- **API Gateway / Proxy**: el BFF redirige cada peticion /api/* al microservicio correspondiente.
- **Database per Service**: cada servicio posee su propia base de datos.
- **API REST**: comunicacion sincrona HTTP/JSON entre capas.

5. Flujo de integracion (ejemplo: compra)

1. El usuario navega el catalogo -> Frontend consulta GET /api/productos al BFF.
2. El BFF redirige a ms-productos, que lee de db_productos via JPA.
3. El usuario agrega productos -> POST /api/carrito/{id}/agregar -> ms-carrito.
4. Al pagar -> POST /api/pagos/crear-preferencia -> ms-pagos -> MercadoPago.
5. ms-bodega descuenta stock y ms-delivery genera el seguimiento de la entrega.